# KinkSol V3

Introductory User Guide

Adam K. Thomasson

28 September 2026

**Document scope**

This guide accompanies the supplied `KinkSol_V3.zip`. It describes how to install and run the Python code, how the model and solver layers fit together, how to write a new model file, and what the scan, solver and output settings actually do. It is a starting guide for users rather than a derivation of any particular numerical theory.

## Contents

1 Getting started 2

2 How the code fits together 2

3 Model configuration and parameter scans 3

4 Creating a new model file 4

5 Solver controls 7

6 Automatic differentiation 9

7 Outputs and saved results 11

# 1 Getting started

## 1.1 Requirements and installation

KinkSol is a Python program: there is no C/C++ build step. Use Python 3.10 or newer and install `numpy`, `scipy` and `matplotlib`. The supplied archive contains the Python modules; the three packages above are the imports needed to run the supplied models. Run each model from the top-level KinkSol directory; the supplied scripts also insert that directory into the import path when invoked as files.

## 1.2 Which model should I run first?

| File in `models/` | Role |
|---|---|
| `model_template_DOMAIN_WALL.py` | Worked three-field Cartesian domain wall; a useful first run. |
| `model_template_SPHALERON.py` | Worked three-field radial-string ansatz. |
| `model_2HDM_DOMAIN_WALL.py` | More involved six-field global 2HDM domain wall, including a branch-sensitive scan seed. |
| `model_template_BLANK.py` | Starting point for a new model. It deliberately raises `NotImplementedError` until its equations, seed and optional output functions are replaced. |

Each worked file defines `main()`: with `variation_mode="none"` it solves one parameter point; with `"single"` or `"multi"` it runs a scan. The default example grids are fairly large. Reducing `grid_shape` provides a quicker programming check, but changes the physical box unless `grid_spacing` changes too. Close interactive plot windows to let the script finish, or set `show_solution_plots=False` and `show_scan_plot=False` under `output`.

**What a successful run reports**

Check `converged=True` and the final infinity norm $\|F\|_\infty$. The solver may return its best iterate with `converged=False`; a smooth plot alone does not demonstrate convergence or physical correctness.

# 2 How the code fits together

The entry point in a model calls `run_single()` or `run_scan()`. `compile_problem()` constructs the grid, assigns a residual function to each row, and records the row's declared dependencies. `ProblemOperators` evaluates the residual and sparse Jacobian. `HybridSolverStrategy` selects the pointwise and global Newton stages; the runner then computes any observable and handles plots and files.

| Folder | Main responsibility |
|---|---|
| `models/` | Physics, parameter choices, boundary conditions, guesses and plots. |
| `core/` | Grid/indexing, row compilation, parameter points, automatic differentiation and sparse operators. |
| `solvers/` | Pointwise Newton, sparse Newton and step-limited trust-region Newton. |
| `strategy/` | Hybrid phase selection, rescue and scan continuation state. |
| `run/` | Single-point/scan entry points and output files. |
| `diagnostics/` | Iteration history and phase timings. |

### 2.1 Grid and field indexing

Only a *one-dimensional* row system is currently compiled, even though some indexing helpers contain 2D/3D utilities. For $m$ fields and $N$ sites, the solver vector has $mN$ entries in complete field blocks:

$$u = [u_0(0), \dots, u_0(N-1), u_1(0), \dots, u_{m-1}(N-1)], \qquad i(a, j) = aN + j.$$

An interior residual receives `field` (this flat vector) and `i` (the *flat row index*). The spatial site is `site = i % n_grid`. `indexer.field_index(a, site)` and `indexer.field_slice(a)` avoid manual offsets. By contrast, `compute_observable()` and `make_solution_plot()` receive the converged field reshaped to `(n_fields, n_grid)`.

For Cartesian grids, $x_j = (j - (N-1)/2)\Delta x$, so an odd $N$ puts a site at zero. For radial grids, $r_j = j\Delta r$; the $r = 0$ residual must be supplied by a suitable boundary condition rather than evaluating singular bulk terms there. `compile_problem()` rejects `n_dims!=1` and requires at least three sites.

### 2.2 Residual rows and the Jacobian

There is one equation row per field and site. The row compiler uses `build_boundary_spec()` at the two ends, `build_bulk_functions()` in the interior, then applies any `build_extra_rows()` overrides. `build_dependency_spec()` says which unknowns each interior row reads; it creates the fixed sparsity pattern of $J_{ij} = \partial F_i / \partial u_j$. The sparse Jacobian omits any dependency you fail to declare, even when the residual itself reads that value. Declaring an extra dependency costs work but does not remove a true Jacobian entry.

## 3 Model configuration and parameter scans

### 3.1 The main configuration keys

Each model's `get_model_config()` returns a Python dictionary. These are ordinary Python values: edits take effect on the next run without a build.

| Entry | Meaning |
|---|---|
| `n_dims` | Set to `1`; higher dimensions are not compiled by the present row system. |
| `geometry` | `"cartesian"` (centred coordinate) or `"radial"` (origin at site zero). |
| `coord_names` | One-element tuple such as `("x",)` or `("r",)`; keep the comma. |
| `grid_shape` | One-element tuple `(N,)` with $N \geq 3$. |
| `grid_spacing` | One-element tuple `(dx,)` with positive spacing. |
| `n_fields` | Number of unknown components at each site; update every model builder if it changes. |
| `parameters` | Base dictionary of physical constants and tunable inputs. |
| `variation_mode` | `"none"`, `"single"`, or `"multi"`; selects the runner in `main()`. |
| `single_variation` | One range used by a single-parameter scan (Section 3.2). |
| `multi_variation` | List of ranges used for a Cartesian-product scan. |
| `observable_name` | Axis label for a scan observable. |
| `solver`, `output` | Nested dictionaries explained in Sections 5 and 7. |

The approximate Cartesian box length is $(N-1)\Delta x$; an example `(101,)` with spacing `(0.2,)` extends from $-10$ to $+10$. Changing $N$ without checking both resolution and box size can select a different numerical problem. For a radial model the outer radius is $(N-1)\Delta r$.

### 3.2 Choosing scan points

With `variation_mode="none"`, the model solves its base `parameters` once. A `"single"` scan supplies one range:

```
"variation_mode": "single",
"single_variation": {
    "name": "beta", "min": 1.0, "max": 2.0,
    "num": 5, "is_log": False,
},
```

This makes five points including both endpoints. For `is_log=True`, `min` and `max` must be *positive values*, not logarithmic exponents. A `"multi"` scan uses a list of such dictionaries in `multi_variation` and takes their Cartesian product; two ranges with four and three samples make 12 points.

At every point the runner copies the base parameter dictionary, inserts the varied values, and calls `finalize_parameters(params)` if the model has it. Use this callback for constraints and derived quantities: in the worked domain wall, `lambda_1=lambda_2=2*beta` is updated whenever `beta` changes. The runner offers the previous *converged* field as a continuation seed; a model can alter it in `build_continuation_guess()`. Scan order can therefore affect which solution branch is found. A failed point records a non-finite scan observable rather than treating its energy as a solution.

## 4 Creating a new model file

Start with a copy of `models/model_template_BLANK.py` and keep its section headings and `main()` entry point. Give the copy a new name, such as `model_my_wall.py`. Then:

1. In `get_model_config()`, set a one-dimensional grid, the field count and the physical parameters.
2. Replace *every* blank bulk equation. Map field indices to the corresponding functions in `build_bulk_functions()`.
3. Supply left and right residuals for every field in `build_boundary_spec(config, params)`.
4. List every value read by a row in `build_dependency_spec()` and the boundary dependency lists; add exceptional rows if necessary.
5. Return a real, flat vector of length `n_fields*n_grid` from `build_initial_guess(config, params)`.
6. Implement the blank `compute_observable()` and `make_solution_plot()` placeholders, or *delete* those optional functions. Change the output switches as desired.

The runner calls `compute_observable` whenever it exists, even if no observable file is requested. Thus leaving its placeholder in the blank template raises an error. The plot placeholder is called if solution plots are enabled. Test a small grid before attempting a large scan.

### 4.1 Residual functions and sparse dependencies

Each bulk function is called as `f(u, i)` after any extra constants have been bound, for example with `functools.partial`. It must return one scalar residual, equal to zero for a solution. The arguments are the field-major flat vector and its *flat row* index. For a cross-field coupling, use `site = i % N` and `a*N + site` (or the indexer) to read field $a$. The built-in boundary helpers in `core/boundary.py` include `dirichlet_zero`, `dirichlet_value`, `neumann_left_1d` and `neumann_right_1d`.

The dependency specification can declare a broad default:

```
return {"default": {"fields": [0, 1, 2], "stencil_radius": 1}}
```

This registers all three fields at sites $j-1, j, j+1$ for every interior row. A more precise alternative lists *relative* site offsets for each bulk field:

```
return {"bulk_by_field": {
    0: {"deps": [(0, -1), (0, 0), (0, +1), (1, 0)]}
}}
```

A plain `"default"` can also supply its own `"deps"` list. Boundary `left_deps` and `right_deps` instead contain *absolute* `(field, site)` pairs. For each boundary, declare the values its callback actually reads.

For an exceptional origin equation, interface condition or constraint, `build_extra_rows()` may return dictionaries with `field`, absolute `site`, `func` and `deps`; these replace the corresponding ordinary rows. For example:

```
return [{"field": 0, "site": 0, "func": origin_eq,
         "deps": [(0, 0), (0, 1)]}]
```

Check that the number of independent equations still matches the unknowns.

### 4.2 Complete one-field example: a Cartesian kink

The following small model solves

$$\phi'' - \lambda(\phi^2 - v^2)\phi = 0, \qquad \phi(-10) = -v, \quad \phi(10) = +v.$$

Save the code as `models/model_my_kink.py` and run `python -m models.model_my_kink` from the project root. The tanh seed selects the expected wall branch. The second derivative uses the centred three-point stencil; the boundary rows fix the endpoints.

```
from functools import partial
import sys
import numpy as np
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from core.boundary import dirichlet_value
from core.grid import build_grid_from_config
from run.run_model import run_single

def get_model_config():
    return {
        "n_dims": 1,
        "geometry": "cartesian",
        "coord_names": ("x",),
        "grid_shape": (101,),
        "grid_spacing": (0.2,),
        "n_fields": 1,
        "parameters": {"v": 1.0, "lam": 1.0},
        "variation_mode": "none",
        "solver": {
            "use_pointwise_phase": False,
            "use_trust_region": False,
            "final_tol": 1e-9,
            "final_max_iter": 30,
            "enable_rescue": False,
        },
        "output": {
            "run_name": "my_kink",
            "show_solution_plots": False,
            "show_scan_plot": False,
        },
    }

def kink_eq(u, i, N, dx, params):
    v, lam = params["v"], params["lam"]
    return ((u[i-1] - 2*u[i] + u[i+1])/dx**2
            - lam*(u[i]**2 - v**2)*u[i])

def build_bulk_functions(config, params):
    grid = build_grid_from_config(config)
    return {0: partial(kink_eq, N=grid.n_grid,
                       dx=grid.grid_spacing[0], params=params)}

def build_boundary_spec(config, params):
    N, v = config["grid_shape"][0], params["v"]
    return {0: {
        "left": partial(dirichlet_value, value=-v),
        "right": partial(dirichlet_value, value=+v),
```

```
        "left_deps": [(0, 0)],
        "right_deps": [(0, N-1)],
    }}

def build_dependency_spec(config, params):
    return {"default": {"deps": [(0, -1), (0, 0), (0, +1)]}}

def build_initial_guess(config, params):
    x = build_grid_from_config(config).build_coordinates()[0]
    v, lam = params["v"], params["lam"]
    return v*np.tanh(v*np.sqrt(lam/2.0)*x)

def main():
    model = sys.modules[__name__]
    config = get_model_config()
    run_single(model, config, dict(config["parameters"]))

if __name__ == "__main__":
    main()
```

The runnable example is deliberately minimal. To turn the *blank* template into a one-field model instead, remove its two unused equation functions, change `n_fields` and its bulk mapping, replace its radial grid and boundaries, and remove or implement its optional placeholders. Its `main()` can then run single points and scans without modification.

### 4.3 Optional observables, plots and continuation

`compute_observable()` returns a scalar and `make_solution_plot()` returns a Matplotlib `Figure`; in both cases `field[a, :]` is the full array for component $a$. Supply any quadrature weights and physical radial measure appropriate to *your* model—the code does not infer an energy functional. `finalize_parameters(params)` may add derived couplings and must return the completed dictionary. When scanning, `build_continuation_guess(config, params, previous_field)` can adapt a previous *reshaped* solution; return a flat vector or a compatible array. If omitted, the previous field is flattened automatically.

## 5 Solver controls

Put the following entries in the `"solver"` dictionary. Unspecified keys take the defaults below from `HybridOptions`; the worked model files explicitly give their own values. The displayed *residual* is the maximum absolute component $\|F(u)\|_\infty = \max_i |F_i(u)|$.

### 5.1 Phase selection and pointwise pre-phase

| Key (default) | Effect |
|---|---|
| `use_pointwise_phase` (`True`) | Run local Newton updates before the global solve. This can help choose a branch but requires extra work. |
| `use_trust_region` (`True`) | Use a step-limited sparse Newton solve with trial steps; `False` uses plain sparse Newton. |
| `pointwise_tol` (`1e-1`) | Residual target for the pointwise pass (also used by a rescue pass). It is *not* the final requested accuracy. |
| `pointwise_max_iter` (`100`) | Maximum pointwise sweeps in the pre-phase. |
| `pointwise_relaxation` (`1.0`) | Multiplier on each local Newton update. Values below one damp it. |
| `pointwise_freq_out` (`1`) | Console reporting interval in pointwise sweeps (also used during rescue). |

The pointwise stage treats neighbours as fixed during each local field update. Even if it misses its own target, the strategy proceeds to the global phase with the returned iterate. In the supplied examples the 2HDM model enables this pre-phase; the other example templates disable it. All four supplied models set `use_trust_region=False`.

### 5.2 Final sparse Newton solve

| Key (default) | Effect |
|---|---|
| `final_tol` (`1e-6`) | Absolute target on the full residual infinity norm. |
| `final_max_iter` (`100`) | Maximum global Newton iterations. |
| `final_freq_out` (`1`) | Console reporting interval for global iterations. |
| `final_step_scale` (`1.0`) | Multiplies the plain sparse Newton step; unused when `use_trust_region=True`. |

A sparse Newton step solves $J(u)\,\Delta u = -F(u)$. Plain mode applies the scaled update directly. The alternate mode limits the Newton step by a radius and tests candidate residuals before accepting one. These options change the *iteration strategy*; they do not change your equations, boundary rows, or declared Jacobian dependencies.

### 5.3 Trust-region-style step controls

These keys matter when `use_trust_region=True`.

| Key (default) | Effect |
|---|---|
| `trust_initial_radius` (`1.0`) | Initial upper bound on the norm of a trial Newton step. |
| `trust_min_radius` (`1e-6`) | Lower limit; stop searching if the radius reaches it without an acceptable step. |
| `trust_max_radius` (`50.0`) | Upper limit when growing the radius. |
| `trust_shrink` (`0.75`) | Factor applied to a rejected trial step and radius. |
| `trust_grow` (`1.5`) | Radius multiplier after a sufficiently good trial. |
| `trust_max_trials` (`10`) | Maximum candidate steps per global iteration. |
| `trust_accept_growth` (`50.0`) | Accept a finite candidate with new residual no larger than this factor times the old residual. Values above one allow temporary worsening. |
| `trust_good_improve` (`0.75`) | Grow the radius if new residual is below this fraction of the old residual. |

The trust mode is a step-limited Newton algorithm with a residual test; it does not compute a classical trust-region model-reduction ratio.

### 5.4 Rescue and near solutions

| Key (default) | Effect |
|---|---|
| `enable_rescue` (`True`) | After a failed global solve, start a pointwise rescue from the best retained iterate and retry the global solve. |
| `rescue_pointwise_max_iter` (`50`) | Maximum sweeps for this rescue stage. |
| `rescue_pointwise_relaxation` (`1.0`) | Local update multiplier during rescue. |
| `accept_best_near_tol` (`False`) | Return the best iterate as a near solution when its residual meets the relaxed bound below. |
| `near_tol_factor` (`10.0`) | A near iterate must satisfy $\|F\|_\infty \le$ `near_tol_factor` $\times$ `final_tol`. |

The returned near iterate is still marked `converged=False`. A scan does not count it as a valid observable point or a new continuation seed, and `save_only_converged=True` skips saved per-point field and summary files for it. Avoid loosening tolerances just to obtain a smooth plot: inspect the residual and check grid convergence.

## 6 Automatic differentiation

### 6.1 How a Jacobian entry is calculated

Newton's method needs $J_{ij} = \partial F_i / \partial u_j$, where $F_i$ is one residual row and $u_j$ is one entry of the flat field vector. The model supplies only $F_i(u)$: `core/autodiff.py` evaluates its callback with a *dual number* in place of an ordinary value. A dual number carries a real value and a derivative coefficient,

$$u_j \longmapsto u_j + \epsilon, \qquad \epsilon^2 = 0, \qquad F_i(u + \epsilon e_j) = F_i(u) + \epsilon J_{ij}.$$

Ordinary arithmetic propagates both parts by the chain rule. For example, if $z = a + b\epsilon$, then $\sin z = \sin a + b\cos(a)\epsilon$. The coefficient of $\epsilon$ returned by the callback is the derivative. The seeded array entry is restored after each evaluation, even if the callback raises an exception. This is forward-mode differentiation, evaluated to floating-point precision; it does not use a finite-difference step.

The compiler first turns the model's `build_dependency_spec()`, boundary dependency lists and special-row dependencies into a fixed sparse pattern. Automatic differentiation calculates derivatives *only* for these declared columns. The values are inserted into a SciPy CSR Jacobian for the global Newton solve. The pointwise pre-phase and rescue also use dual numbers, but differentiate only the same-site field block while holding neighbouring sites fixed.

For the kink equation in Section 4.2, the interior residual is

$$F_i = \frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2} - \lambda(u_i^2 - v^2)u_i.$$

The three declared derivatives are $J_{i,i-1} = J_{i,i+1} = 1/\Delta x^2$ and

$$J_{i,i} = -\frac{2}{\Delta x^2} - \lambda(3u_i^2 - v^2).$$

The model author writes the residual and its dependency list; the code obtains these derivative values automatically.

## 6.2 Operations supported by the dual-number class

The support is deliberately limited to the following operations in `core/autodiff.py`. They may be combined and nested in a model's residual callback.

| Group | Supported operations |
|---|---|
| Python arithmetic | Addition, subtraction, multiplication, true division, power (`**`), and unary plus/minus, including a real value on either side of a dual number. |
| NumPy arithmetic ufuncs | `np.add`, `np.subtract`, `np.multiply`, `np.divide`, `np.power`. |
| NumPy elementary ufuncs | `np.sin`, `np.cos`, `np.exp`, `np.sqrt`, `np.tanh`, `np.log`. |

For a dual argument, `np.sqrt` and `np.log` require a *strictly positive* real part. If the exponent is dual, a power also requires a strictly positive base; the same restriction applies to a real base raised to a dual exponent. Ordinary real powers still have their usual domain restrictions. This implementation supports ordinary ufunc calls, not arbitrary NumPy functions or ufunc reductions.

For example, `np.abs`, `np.sinh`, `np.cosh`, `np.arctan`, `np.maximum` and `np.where` are *not* implemented for `Dual`. Neither are general comparisons or an `abs(Dual)` method. `math.sin`, conversion with `float()`, or forcing `dtype=float` inside a residual may fail or discard the derivative coefficient. When possible, rewrite expressions using supported operations; for instance, $\sinh z = (\exp z - \exp(-z))/2$. For a genuinely new operation, add its chain rule to `Dual` and verify it against finite differences. Take particular care with piecewise formulas and points where a derivative does not exist.

### 6.3 Choosing scalar, row or automatic mode

Set `"jacobian_autodiff"` at the *top level* of `get_model_config()`, alongside `"solver"`:

```
"jacobian_autodiff": "auto",
```

| Value | Behaviour |
|---|---|
| `"scalar"` | Seed one declared variable at a time. A row callback is called once per declared derivative. |
| `"row"` | Seed a vector of coefficients for the row's declared variables and obtain them in one callback. In the sparse Jacobian, rows with fewer than seven dependencies still use scalar mode. |
| `"auto"` (default) | Try row mode for sparse rows with 7–64 dependencies and for pointwise blocks with 5–64 fields; otherwise use scalar mode. On its first use for each callback and derivative-vector length, compare row and scalar results, and cache the choice. |

In `"auto"`, a vector-mode error or a disagreement with scalar automatic differentiation selects scalar mode for that callback and width. This is *not* a fallback to numerical differentiation. Forcing `"row"` does not make an unsupported operation work. Also, both modes can return a wrong derivative if a callback drops the dual part and returns an ordinary number: the differentiation helper then returns zero. The initial mode comparison cannot detect an error shared by both modes or guarantee every later branch of a conditional. After writing a new equation, compare representative Jacobian columns with finite differences on a small, perturbed grid. If vector propagation is incompatible, `"scalar"` is a useful diagnostic; it does not replace that check.

## 7 Outputs and saved results

Set options in the `"output"` dictionary. Missing entries take the defaults in `run/output.py`. The runner makes the folders `<base_dir>/<run_name>/points/`, `plots/`, and `scan/`. A model with `base_dir` set to a project-root absolute path writes there regardless of the current working directory.

| Location and naming key (default) | Effect |
|---|---|
| `base_dir` (`"outputs"`) | Parent output path. A relative path is resolved from the shell's working directory. |
| `run_name` (`"default_run"`) | Subdirectory for this run; choose a different name to keep runs apart. |
| `file_tag` (`None`) | Optional extra filename suffix, e.g. for an experiment label. |

| Console key (default) | Effect |
|---|---|
| `print_point_summary` (`True`) | Print a result and phase summary for each solved point. |
| `print_scan_summary` (`True`) | Print the overall scan summary. |
| `print_params_each_point` (`True`) | Display each parameter dictionary as the scan advances. |
| `print_output_paths` (`True`) | Print names of files written by the runner. |

| Plot key (default) | Effect |
|---|---|
| `show_solution_plots` (`True`) | Display a model's solution figure interactively for each point. |
| `save_solution_plots` (`False`) | Save that figure under `plots/`. Requires `make_solution_plot`. |
| `show_scan_plot` (`True`) | Display the runner's observable scan plot. Applies to scans only. |
| `save_scan_plot` (`False`) | Save the scan plot under `plots/`. |
| `plot_format` (`"png"`) | Filename extension/format for saved figures. |
| `plot_dpi` (200) | Resolution for saved raster figures. |

| Saved data key (default) | Effect |
|---|---|
| `save_point_summary` (`False`) | Write a per-point JSON result under `points/`. |
| `save_scan_summary` (`False`) | Write a summary under `scan/`; CSV when concise, JSON otherwise. |
| `save_observables` (`False`) | Write scan parameters and observable values to CSV under `scan/`. |
| `save_final_field` (`False`) | Write a space-delimited text field under `points/`. |

| Save style key (default) | Effect |
|---|---|
| `concise` (`True`) | Shorten per-point JSON metadata; select CSV for scan summary. |
| `save_only_converged` (`True`) | Save per-point files and plots only for converged solves; scan summaries still include failures. |

Saved fields have one row per site and one column per field (`field.T`); coordinate values are *not* included. Reconstruct them from the grid settings. A failed scan point has a blank observable in CSV, and the scan summary records its failure. For a single point, the runner calls `compute_observable` if it exists even on failure; check `converged` before using that value.

For an unattended run, this is a useful minimal override:

```
"output": {
    "run_name": "small_grid_check",
    "show_solution_plots": False,
    "show_scan_plot": False,
    "save_point_summary": True,
    "save_final_field": True,
}
```